

Backend + ML/AI Engineer Roadmap

Starting point: CS50 ✓, CS50P (in progress), CCC ~J5/S2, a few games, a few GPT wrappers

Target: ML Engineer / Backend+AI Engineer — builds and ships ML systems end-to-end

Stack: Python · FastAPI · PostgreSQL · Redis · React (TypeScript) · Docker · GitHub Actions · PyTorch · standard ML/AI libs

Pace: ~8-12 hrs/week during school, more in summers · Summer after Gr. 10 → Summer after Gr. 12

How-to-learn shorthand

Label	What it means
Do-first	Attempt cold before reading or watching. Struggle is the point.
Type-along → variant	Follow the tutorial by typing (not pasting), then build a small variant from memory.
Read → implement	Read one section, close it, implement the core idea from scratch. Never read two sections back-to-back without implementing.
Watch → code same day	Watch, then write code applying the concept before the day ends.
Timed reps	Do it under real time pressure with no hints open. Review only after a genuine attempt.
Spaced re-solve	Re-solve the same problem cold ~1 week later without notes.
AI-assisted	Use AI to get unstuck or explore variants — but write code yourself first. Never have AI produce the first draft.
Read-only	Reference or orientation material only — not a skill to produce.

Effort Allocation (across all years)

Phase	Area	%	Est. Duration
0	DSA & Competitive Programming	5%	Ongoing — 1 session/week forever

Phase	Area	%	Est. Duration
1	Programming Fundamentals	6%	4–5 weeks
2	Backend Engineering	20%	11–13 weeks
3	DevOps & Cloud — lean slice	4%	3–4 weeks
4	Applied AI / LLMs	10%	6–7 weeks
5	System Design + ML Systems	6%	4 weeks
6	DevOps & Cloud — ML infra slice	4%	3–4 weeks
7	Classical Machine Learning	13%	8–10 weeks
8	Deep Learning	13%	8–10 weeks
9	MLOps + LLM deepening	8%	5–6 weeks
10	Capstone Projects	11%	9–11 weeks (overlaps)
—	Side Fields	~4%	Ongoing

DSA (Phase 0) runs as one weekly session layered on top of every other phase — it never blocks anything and never finishes. Everything else is sequential within each grade period.

Phase 0 — DSA & Competitive Programming (ongoing, all years)

Target: CCC Senior top 10% — reliable S1–S4, partial S5 $\approx$ USACO Gold. CCO possible with consistent weekly practice over 2+ years.

Structure: One session per week, alternating between (a) a timed CCC past paper and (b) one new Gold-level topic + 2–3 problems. Timed practice moves the needle; topic study alone doesn't.

C++: Switch for all contest work once you finish the Gr. 10 phase. Python TLEs on Gold-level problems — the switch is worth making early.

LeetCode: Skip for now. Pick up NeetCode 150 a few weeks before your first internship application.

Topic	Already solid?	Resource	How to learn
Two pointers, sliding window, hashing, recursion, sorting	Yes	USACO Guide Silver — skim	Timed reps on 1–2 problems, then spaced re-solve.
Trees & BSTs	Partially	USACO Guide Silver — trees	Read → implement each traversal from scratch.
BFS / DFS, graph traversal	Developing	USACO Guide Silver — graphs	Read → implement BFS and DFS without reference. Timed reps on Silver graph problems.
Dijkstra, MST (Kruskal/Prim), topological sort	New	USACO Guide Gold — graphs	Read → implement each from scratch. AI-assisted for Dijkstra edge cases.
Dynamic programming (1D/2D → tree DP, bitmask DP)	New	USACO Guide Gold DP; <i>Competitive Programmer's Handbook</i> (free PDF)	Do-first — 30 min minimum before the editorial. Spaced re-solve 1 week later.
Greedy (exchange argument)	New	USACO Guide Gold — greedy	Read → implement.
Segment trees & Fenwick trees	New	USACO Guide Gold — "Range Queries"	Read → implement from scratch. Fenwick tree before segment trees.
Number theory (modular arithmetic, GCD, primes, combinatorics)	New	<i>Competitive Programmer's Handbook</i> — number theory chapters	Read → implement each primitive (fast exp, sieve, modular inverse).
Algorithm reference	—	cp-algorithms.com	Read-only — look up when you hit something unfamiliar.
Contest practice	—	CEMC official CCC/CCO past papers (cemc.uwaterloo.ca)	Timed reps every other week. Editorial only after a genuine attempt;

Topic	Already solid?	Resource	How to learn
			spaced re-solve afterward.
Online judge	—	DMOJ CCC archive	Timed reps + spaced re-solve for problems that needed an editorial.

Summer after Gr. 10

Focus: programming foundations, backend basics, and the DSA habit. End of summer target: one deployed API and C++ contest-ready.

Summer after Gr. 10 — Phase 1: Programming Fundamentals (6%, ~4-5 weeks)

Assumed solid from CS50/CS50P: core Python, exceptions/file I/O, basic OOP, Git, SQL, HTML/CSS/Flask, API calls/JSON. Everything below is new.

Do C++ first (or alongside finishing CS50P) — needed for Phase 0 contest work.

Topic	Resource	How to learn
C++ for competitive programming — syntax, STL (vectors, maps, sets, priority queues), cin/cout, pointers, basic memory model	<i>Competitive Programmer's Handbook</i> ch. 1-2; learncpp.com	Type-along → variant through learncpp.com linearly. Timed reps on DMOJ in C++ immediately after basics. Target: Silver problems fluently in C++ within 3-4 weeks.
Finish CS50P	CS50P (in progress)	Do-first on every problem set.
Intermediate Python — comprehensions, generators, decorators, context managers, type hints, venv/poetry	Real Python; Corey Schafer Python playlist	Type-along → variant — after the decorator tutorial, write your own timing decorator from scratch.
OOP in depth — inheritance vs. composition, basic SOLID	Corey Schafer "Python OOP"	Type-along → variant — rebuild each class from memory, then write a second for a different domain.
Git for real — branching, merging, conflict resolution, PR workflow	<i>Pro Git</i> (free, git-scm.com/book)	Read → implement. Deliberately create and resolve a merge conflict.
CLI & Linux basics — filesystem, permissions, piping, bash scripting, SSH	MIT Missing Semester (free)	Do-first on every exercise. Use your actual terminal.
Testing — pytest, TDD basics, mocking	<i>Test-Driven Development with Python</i> — Percival (free)	Type-along → variant — write the test before the implementation every time.

🔧 **Project — CLI habit tracker:** Command-line tool (`habits add`), (`habits log done`), (`habits show --week`). SQLite storage, type hints throughout, 10+ pytest tests covering core logic, timing decorator, meaningful Git history. No web framework.

Summer after Gr. 10 — Phase 2: Backend Engineering (20%, ~11-13 weeks)

Start with CS50W — covers Flask, Django, SQL, JS, APIs, and security in one course with real project reps. Layer FastAPI on top afterward for ML serving. Roadmap.sh Backend map

is a useful visual checklist (priority nodes: Internet → VCS → Relational DBs → APIs → Auth → Caching → Testing → CI/CD).

Topic	Resource	How to learn
Full-stack foundations — Flask, Django, SQL, JS, APIs, security	CS50W	Do-first on every project.
HTTP — methods, status codes, headers, statelessness	MDN HTTP docs	Read-only + experiment: use <code>curl</code> on a live API and read the raw responses.
FastAPI — routing, Pydantic models, dependency injection, async	FastAPI official tutorial	Type-along → variant — after each section, rebuild the endpoint for a different resource. AI-assisted for Pydantic errors.
Async & background tasks — <code>async</code> / <code>await</code> , BackgroundTasks, Celery	FastAPI async docs; Celery quickstart	Type-along → variant — get one background task working end-to-end, then add one of your own design.
PostgreSQL — joins, indexes, transactions, normalization, schema design	SQLBolt → pgexercises.com → PostgreSQL tutorial	Timed reps on SQLBolt and pgexercises. Schema design: do-first.
SQLAlchemy ORM — models, migrations, queries	SQLAlchemy docs	Type-along → variant — build a model set, then rebuild for a different domain. AI-assisted for Alembic migration errors.
Redis — caching, sessions, pub/sub	Redis "Introduction to Redis"	Type-along → variant — implement a cache-aside pattern in Python.
Auth — bcrypt, JWT, OAuth2	FastAPI security docs	Type-along → variant — implement full login + protected route, then rebuild from scratch. AI-assisted to reason through the OAuth2 flow.
API testing — pytest + httpx/TestClient	FastAPI testing docs	Do-first — write tests before endpoints on at least one project.
Web security basics — OWASP Top 10, CORS, HTTPS	OWASP docs; MDN CORS	Read-only. Hands-on in the App Security side field.

Grade 11

Focus: ship things, build the Applied AI layer on top of the backend foundation, be useful at hackathons. End of year target: one deployed RAG app in portfolio, internship application ready.

Gr. 11 — Phase 3: DevOps Lean Slice (4%, ~3-4 weeks)

Done first so everything from here can actually ship. Roadmap.sh DevOps map: Docker → CI/CD → Cloud Providers (basics) → Monitoring (basics).

Topic	Resource	How to learn
Docker — images, Dockerfile, docker-compose	Docker "Get Started"	Type-along → variant — containerize the finance tracker from last summer, then deliberately break something and debug it.
CI/CD pipelines	GitHub Actions docs	Type-along → variant — get a pipeline running, then add a step you designed (linting, test coverage threshold). AI-assisted for YAML errors.
First real deployment	Render.com or Railway.app	Do-first — try without a tutorial. Fix what breaks.
CORS & environment variables	MDN CORS; FastAPI CORS middleware docs	Read → implement when wiring your API to a frontend caller.
Structured logging	Python <code>logging</code> + structlog	Type-along → variant — add structured JSON logging to your existing API the day you read about it.

🔧 **Project — Deploy the finance tracker:** Docker + docker-compose (app + Postgres + Redis). GitHub Actions blocks merge on test failure. Deployed on Render/Railway with no

secrets in the repo. Structured JSON logging on every request (user ID, endpoint, response time).

Gr. 11 — Phase 4: Applied AI & LLMs (10%, ~6-7 weeks)

Beyond API calls — the goal is AI-powered backend systems, not just prompt-and-print.

Topic	Resource	How to learn
Prompt engineering — chain-of-thought, few-shot, XML tags, system prompts	Anthropic prompt engineering docs (docs.claude.com)	Read → implement after each technique. Keep a log of what worked.
Structured outputs & function calling — reliable JSON, tool use	Anthropic + OpenAI function calling docs; Instructor library	Type-along → variant — get structured JSON working reliably, then design your own schema for a different use case.
Embeddings & vector DBs — pgvector, Chroma	Hugging Face NLP Course (free)	Type-along → variant — embed a small doc set in pgvector, run similarity queries, swap embedding models and compare.
RAG pipelines end-to-end	DeepLearning.AI "LangChain for LLM App Dev" + "Building and Evaluating Advanced RAG" (free, ~1-2 hrs each)	Type-along → variant — get the tutorial pipeline working, then rebuild for a domain you chose. AI-assisted to diagnose retrieval failures.
RAG evaluation — recall, precision, faithfulness, answer relevance	RAGAS docs; DeepLearning.AI "Building and Evaluating Advanced RAG"	Do-first — define what "good retrieval" means before reading the metrics. Then instrument with RAGAS.
Agents & tool use — multi-step LLM workflows, ReAct	DeepLearning.AI "Building Agentic RAG"; LangChain/LlamaIndex docs	Do-first after the tutorial — design and build an agent for a task you defined.
Agentic frameworks — LangGraph stateful graphs	LangGraph docs + tutorials	Type-along → variant — build the tutorial graph, then design one with a loop and a human-in-the-loop step.
LLM observability — tracing, logging prompts/responses	LangSmith or Langfuse (open-source)	Type-along → variant — instrument one pipeline. You can't debug what you can't observe.

Topic	Resource	How to learn
AI security — prompt injection, data leakage in RAG, API key hygiene	Anthropic safety docs; OWASP LLM Top 10	Read → implement — audit your own RAG app for the top 3 risks.

🔧 **Project — Document Q&A API with RAG:** Authenticated users upload PDFs/plain text → chunks stored as embeddings in pgvector. `/ask` endpoint retrieves relevant chunks, passes to LLM, returns a cited answer. FastAPI + PostgreSQL + pgvector, RAGAS eval scores logged, LangSmith/Langfuse tracing on every request, JWT auth, Docker + GitHub Actions, deployed. This is the internship portfolio piece.

Gr. 11 — Hackathon Backend Toolkit

Role: backend. The AI engineer owns LLM integration; the frontend person owns UI. Your job is the middle layer that doesn't fall over.

Core contributions:

1. **API scaffolding (first 2 hrs):** Stand up FastAPI with CORS, env var handling, and `/health`. The AI engineer gets a base URL immediately and you iterate from there.
2. **Async endpoints:** Every LLM-calling endpoint must be `async`. Use BackgroundTasks or Celery for anything that takes >2s — the frontend should never hang waiting on a model call.
3. **Data model (first 3 hrs):** Simple Postgres schema in docker-compose so everyone shares one connection string. Prevents the "we're all using different local SQLite files" disaster.
4. **ML endpoint if genuinely needed:** Only if the problem requires a real model (spam detection on structured features, a numeric classifier) — not for tasks an LLM handles. Use Hugging Face `pipeline()` or a pre-fitted sklearn model. Never train at the hackathon.

Prep once, reuse every time: Keep a FastAPI starter template with CORS, `/health`, env loading, docker-compose (Postgres + Redis), and GitHub Actions. Takes 30 min to set up once and saves 2 hours at every event.

Don't: Build a full RAG pipeline (that's the AI engineer's job), train a model from scratch, or hardcode anything a teammate needs to change to run locally.

Gr. 11 — Before Internship Applications (fall of Gr. 11)

Applications go out in **fall of Gr. 11** for summer internships. Have these ready:

- **Finance tracker API** — FastAPI, PostgreSQL, auth, testing, CI/CD, Docker, deployed
 - **Document Q&A API** — RAG, pgvector, LLM integration, eval, tracing, deployed and documented
 - **One open-source contribution** — even a documentation fix. Reading someone else's production codebase is a skill you can't build by only building your own projects.
 - **NeetCode 150** — start 3-4 weeks before applications, not now.
-

Summer after Gr. 11

AI internship. Treat everything you observe as Phase 4/5 material happening in real time — production RAG systems, real eval pipelines, actual model serving. Note every gap it exposes; those become Gr. 12 priorities.

Grade 12

Focus: ML depth that makes you genuinely useful — not just someone who calls APIs, but someone who understands what's inside them, can train and serve models, and owns the full experiment cycle.

Gr. 12 — Phase 5: System Design & ML Systems (6%, ~4 weeks)

Runs alongside Phase 6 (Classical ML) early in Gr. 12. Roadmap.sh reference: "Building For Scale" and "Software Design & Architecture."

Topic	Resource	How to learn
Core concepts — load balancing, caching, CAP theorem, replication	System Design Primer (free, GitHub)	Read → implement — after each concept, explain it in your own words. If you can't, you don't know it yet.
Scaling patterns — sharding, message queues (Kafka/RabbitMQ), CDNs	ByteByteGo YouTube	Watch, then draw the architecture from memory before the next video. No drawing = no retention.
Deep dive	<i>Designing Data-Intensive Applications</i> — Kleppmann	Read → implement. One chapter per sitting.
ML systems design — training pipelines, feature stores, model registries, serving infra	<i>Designing Machine Learning Systems</i> — Chip Huyen	Read → implement. The ML-specific extension that general system design books miss entirely.
Practice	<i>System Design Interview</i> — Alex Xu Vol. 1: URL shortener, rate limiter, chat app; then design a model serving system	Do-first — full design on paper before reading Xu's chapter. AI-assisted to stress-test ("what breaks at 10M users?") afterward.

🔧 **Project — Architecture doc:** Two-page design doc: (1) how the Gr. 11 Document Q&A API scales to 100k users; (2) how the serving layer changes if it runs a fine-tuned model instead of an API call. Two constraint sets, one document. No code — just reasoning using the vocabulary from this phase.

Gr. 12 — Phase 6: Classical Machine Learning (13%, ~8-10 weeks)

CS50 AI is the spine. Start it at the beginning of Gr. 12 and let it carry through the first half.

Roadmap.sh ML map: Supervised Learning, Unsupervised Learning, and Model Evaluation sections.

Topic	Resource	How to learn
Survey foundation — search algorithms, classical ML, intro neural nets	CS50 AI (start here)	Do-first on every project — implementing minimax, naive Bayes yourself is where the learning is.
Python ML stack — NumPy, Pandas, Matplotlib, seaborn	Kaggle Learn micro-courses (free)	Type-along → variant — reproduce key operations on a different dataset from memory.
ML workflow — train/test split, cross-validation, metrics (precision/recall/AUC)	Kaggle Learn "Intro to ML" + "Intermediate ML"	Type-along → variant — implement a full pipeline from scratch after the tutorial, not during it.
Supervised learning — linear/logistic regression, decision trees, random forests, gradient boosting (XGBoost/LightGBM)	<i>Hands-On ML with Scikit-Learn, Keras & TensorFlow</i> — Géron, Part 1	Read → implement one algorithm per session. AI-assisted to understand why accuracy differs, not to fix code.
Feature engineering	Kaggle Learn "Feature Engineering"; Géron ch. 2	Read → implement.
Unsupervised learning — k-means, PCA, UMAP	Géron, relevant chapters	Implement, then apply to a dataset you chose.
Experiment tracking — log hyperparameters, metrics, artifacts	Weights & Biases (W&B) quickstart (free)	Instrument every training run from this point forward. Habit matters more than the tool.
Model interpretability — SHAP values	SHAP library docs; Kaggle "Explainability"	Read → implement — run SHAP on your Kaggle competition model.
Optional	Andrew Ng "Machine Learning Specialization" (Coursera)	Watch → code same day if you use it.
Kaggle practice	Titanic → tabular playground competition	Do-first — baseline submission before reading

Topic	Resource	How to learn
		any public notebooks. Incorporate one idea at a time.

🔧 **Project — ML Prediction API:** Train a gradient boosting model on a real tabular dataset (housing prices, loan default, or churn — enough rows that preprocessing decisions matter). Pipeline: data cleaning → feature engineering → model selection (3+ algorithms logged in W&B) → final model → SHAP report. Serve via FastAPI (`/predict`): JSON features in, prediction + confidence + top 3 SHAP features out. Pytest tests, Docker, basic drift monitoring, deployed.

Gr. 12 — Phase 7: Deep Learning (13%, ~8-10 weeks)

Complete derivative/chain rule in school math before hitting backprop. CS50 AI's neural networks and language weeks are the primer.

Roadmap.sh ML map: CNNs, RNNs, Transformers, Autoencoders.

Topic	Resource	How to learn
Neural net fundamentals — forward/backward pass, backprop	3Blue1Brown neural network series (YouTube)	Watch → code same day — implement a 2-layer net forward and backward pass in pure NumPy. Highest-leverage exercise in this phase.
PyTorch — tensors, autograd, <code>nn.Module</code> , training loops, data loaders	PyTorch "60 Minute Blitz" + tutorials	Type-along → variant — rebuild the network for a different task from memory. AI-assisted for tensor shape errors.
Top-down approach	fast.ai "Practical Deep Learning for Coders" (free)	Do-first on every notebook — run it, break something intentionally, fix it.
Transfer learning — pre-trained weights, fine-tuning	fast.ai; Hugging Face docs	Type-along → variant — fine-tune on the tutorial dataset, then on a small custom dataset.
CNNs	fast.ai; d2l.ai CNN chapter	Implement on a dataset you chose, not the one fast.ai uses.
Transformers — attention, encoder/decoder; Karpathy's "Let's build GPT"	3Blue1Brown "Attention"; d2l.ai transformer chapter; Andrej Karpathy (YouTube)	Watch → code same day for 3B1B. Implement scaled dot-product attention in PyTorch before using <code>nn.MultiheadAttention</code> .
Experiment tracking for DL	W&B PyTorch integration docs	Log every training run. Same habit, different context.
Optional	<i>Dive into Deep Learning</i> (free, d2l.ai)	Read → implement every chapter's notebook, then modify.

🔧 **Project — Fine-tuned classifier served as an API:** Classification task with a real use case (e.g., urgent/non-urgent support tickets, review aspect detection). Fine-tune a small pre-trained transformer (DistilBERT via Hugging Face) on 500–1000 labelled examples. Full W&B training run (loss curves, eval metrics per epoch, hyperparameter sweep over at least LR and batch size). FastAPI endpoint: raw text in, predicted class + confidence out. Docker + deployed.

Gr. 12 — Phase 8: DevOps Cloud Slice + MLOps (8%, ~5-6 weeks)

Runs at the end of Gr. 12 alongside the Capstone. Roadmap.sh DevOps map: Cloud Providers, Infrastructure Monitoring, Container Orchestration (basics).

Topic	Resource	How to learn
Cloud deployment on AWS — Elastic Beanstalk or App Runner, IAM basics	AWS "Getting Started" guides	Type-along → variant — deploy an existing API to AWS, then modify the config. AI-assisted for IAM errors.
Managed ML platforms — training endpoints, batch inference, managed serving	AWS SageMaker or GCP Vertex AI (pick one)	Type-along → variant — deploy a model end-to-end, then modify the serving config.
GPU instances — GPU vs. CPU tradeoffs, spot instances	AWS EC2 GPU docs; Lambda Labs	Read-only + one experiment: run a small training job on a spot instance.
MLOps — model serving, versioning, monitoring drift	Made With ML — Goku Mohandas (free, madewithml.com)	Type-along → variant — follow the serving tutorial, then add a drift-detection check of your own design.
Lightweight fine-tuning at scale — LoRA/PEFT	Hugging Face PEFT docs	Type-along → variant — fine-tune with LoRA, then compare training time and performance against full fine-tuning from Phase 7.
LLM evaluation — building real evals	RAGAS (revisited); DeepLearning.AI "Building and Evaluating Advanced RAG"	Do-first — define what "good" means for your Phase 4 RAG app, then instrument it.
Data pipelines for ML — batch ETL feeding a training run	DataTalksClub "Data Engineering Zoomcamp" — first two weeks only (free)	Type-along → variant — get the tutorial DAG running, then build one for a data source you chose.

🔧 **Project — Redeploy on AWS + MLOps layer:** Take the Phase 6 ML Prediction API, redeploy on AWS (SageMaker or App Runner). Add: a scheduled Airflow DAG pulling fresh data daily with basic drift detection; a W&B dashboard showing model performance over

time; a GitHub Actions workflow that retrains and redeploys if drift exceeds a threshold.
Full loop: data → train → eval → deploy → monitor → retrain.

Gr. 12 — Capstone: Agentic Assistant (11%, ~9-11 weeks, overlaps Phase 8)

The hardest project. Do it last — it pulls from everything.

Project — LangGraph multi-step agent with FastAPI backend: Build an agent that solves a problem you actually care about. Good examples: a research assistant that searches the web and stores findings by topic in Postgres; a coding assistant that reads a GitHub issue, proposes a fix, and drafts a PR; a study tool that generates questions, evaluates your answers, and adapts difficulty.

Stack: LangGraph (at least one loop, one conditional branch), FastAPI, PostgreSQL for state persistence between sessions, LangSmith/Langfuse tracing on every agent step, an eval harness against 20+ handwritten test cases, Docker + GitHub Actions + deployed on AWS.

How to learn: Draw the full agent graph (nodes, edges, loops, failure modes) on paper before writing any code. Write the eval harness before the agent logic — what does success look like? AI-assisted to stress-test failure modes and design the eval harness; this is one place where using AI to design (not implement) is appropriate.

Summer after Gr. 12

Internship (if not secured in summer after Gr. 11), independent project, or research before university. The plan above sets up cleanly for CS, Statistics, or Data Science — the backend work grounds core CS courses; the ML engineering work puts you ahead in ML/AI courses; experiment tracking habits read well to research labs.

What to look for in a program: strong ML faculty, ML systems and probabilistic ML courses (not just intro ML), and co-op/internship integration.

Side Fields (~4%, each <1%, ongoing)

None of these block anything.

Field	Resource	How to learn	Project / Outcome
App Security	PortSwigger Web Security Academy (free)	Do-first on every lab — attempt to exploit before reading the hint.	Find and fix one real vuln in your own deployed API before the internship application.
Frontend (React + TypeScript)	Scrimba "Learn React" (free); TypeScript docs	Type-along → variant — build the tutorial component, then rebuild from memory for a different use case.	A minimal frontend for the Gr. 11 Document Q&A API — just enough to demo without Postman.
Networking basics	<i>Computer Networking: A Top-Down Approach</i> — Kurose & Ross, ch. 1-3 only	Read-only	Understand what happens when your RAG app makes an API call; useful for debugging latency.
Technical writing	Google's free "Technical Writing for Engineers"	Read → implement for a real project	Proper README + OpenAPI docs for one deployed project.
Open source contributing	Any RAG or agent library you're already using (LangChain, LlamaIndex, RAGAS)	Read the codebase, find a real issue — even a documentation fix	One merged PR. Reading someone else's production codebase is a skill you can't build by only building your own projects.

Stack Summary

Layer	Choice	Why
Language	Python	Already know it; ML/AI ecosystem is Python-native
API framework	FastAPI	Async, typed, OpenAPI-native, production-grade, ML-serving ready

Layer	Choice	Why
Database	PostgreSQL + pgvector	Relational default + vector search without a separate DB
Caching / queuing	Redis + Celery	Standard backend pattern; needed for ML pipelines too
ORM	SQLAlchemy	Pairs with FastAPI cleanly, mature ecosystem
Frontend	React + TypeScript	Typed, component-based, industry standard
Classical ML	scikit-learn + XGBoost/LightGBM	Sklearn for workflow, XGB/LGBM for production tabular
Deep Learning	PyTorch	Industry standard for research and production
LLM / AI libs	LangChain, LangGraph, LlamaIndex, Hugging Face	Standard applied AI stack
Experiment tracking	Weights & Biases (W&B)	Best UX; free tier covers everything in this roadmap
Containerisation	Docker + docker-compose	Local dev parity with production
CI/CD	GitHub Actions	Lightweight, free, integrates with everything
Deployment	Render / Railway → AWS	Low friction first, scale up later
Cloud AI	AWS SageMaker or GCP Vertex (pick one)	Managed training + serving for Gr. 12 phases

How to Actually Learn from This Material

Courses (CS50P, CS50W, CS50 AI, fast.ai): Do the problem set yourself before looking at any solution. The struggle before the answer is the learning.

Docs and tutorials (FastAPI, USACO Guide, SQLBolt): Type the code yourself. After each section, close it and build a small variant from memory. Following along without closing the tab is tutorial hell — feels productive, doesn't build the skill.

Intuition videos (3Blue1Brown, StatQuest): Immediately after watching, do one hands-on exercise applying the concept. Same day, always.

Books (Géron, Kleppmann, Competitive Programmer's Handbook): Read a chapter, close it, implement or explain the core idea from memory before checking yourself.

Competitive programming: Timebox each problem before the editorial. When you do read it, close it and reimplement from scratch. Revisit problems you struggled with a week later.

General:

- If you can't explain a concept without notes, you don't know it yet.
- Keep a short weekly build log — what you built, what broke. Feeds your portfolio and surfaces retention gaps.
- From Phase 6 onward: log every training experiment in W&B. The habit matters more than the tool.
- Commit code regularly. Shipping something imperfect teaches more than polishing something you never finish.

Using AI: Write your first attempt before asking for help. AI is useful for explaining errors, reviewing code you already wrote, generating edge cases, and explaining concepts differently. What undermines the learning: pasting a project description and asking for the solution.